

02 Matrices

2.1 Matrices

Rows, Columns, Indices, and the Starting Point of Tensors

Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>



Core Skills You Will Master

1

Understand what a matrix is: a two-dimensional array arranged in rows and columns, used both to store data and to represent linear transformations

3

Use NumPy fluently to extract row vectors, column vectors, and single elements from a matrix; understand the basic idea of block matrices

5

Master matrix addition and subtraction: two matrices of the same shape are combined element-wise, satisfying the commutative and associative properties

7

Connect everything through the lens of tensors: a scalar is a 0th-order tensor, a vector is 1st-order, a matrix is 2nd-order, and color images and video are higher-order tensors

2

Master row/column index conventions: row index increases top to bottom, column index increases left to right; math counting starts at 1, NumPy indexing starts at 0

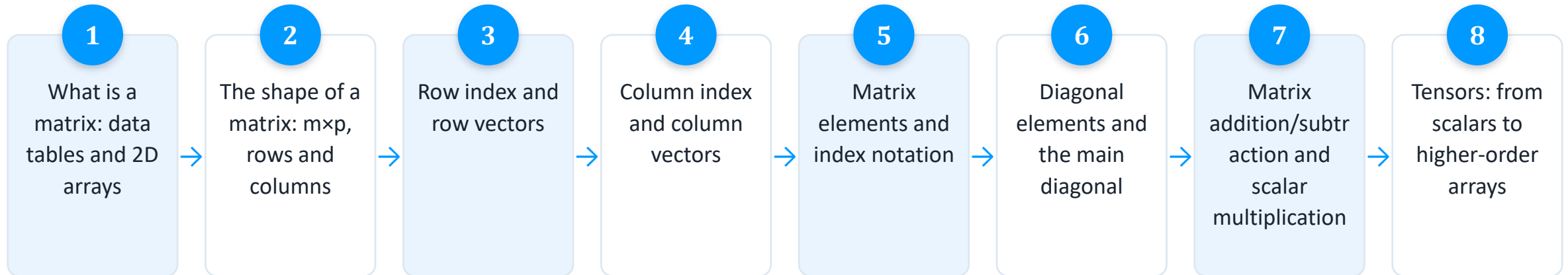
4

Understand diagonal elements and the main diagonal: the special set of entries whose row index equals its column index

6

Master scalar multiplication of a matrix: scaling every entry by a single scalar

How This Section Unfolds



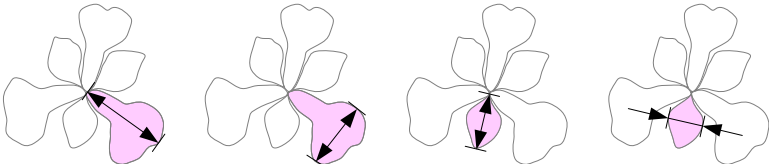
What Is a Matrix?

A matrix is a two-dimensional array arranged in rows and columns

- Intuitively, a matrix is just a table full of numbers — like a student's grade sheet, where each row is a student and each column is a subject
- A matrix is not only a mathematical concept; it shows up everywhere in everyday data: images, tables, sensor readings can all be represented as matrices
- This section starts with two examples — the Iris dataset and photo pixels — to see concretely how a matrix organizes data

Figure 1 · The Iris Dataset Is Also a Matrix

- Each row is one iris sample; the first 4 columns are sepal/petal length and width, the last is the species
- 150 samples, 3 species — strip the header and it's simply a 150×4 data matrix



Index	Sepal length X_1	Sepal width X_2	Petal length X_3	Petal width X_4	Species C
1	5.1	3.5	1.4	0.2	Setosa C_1
2	4.9	3	1.4	0.2	
3	4.7	3.2	1.3	0.2	
...	...	...	...	...	
49	5.3	3.7	1.5	0.2	
50	5	3.3	1.4	0.2	Versicolor C_2
51	7	3.2	4.7	1.4	
52	6.4	3.2	4.5	1.5	
53	6.9	3.1	4.9	1.5	
...	...	...	...	...	
99	5.1	2.5	3	1.1	Virginica C_3
100	5.7	2.8	4.1	1.3	
101	6.3	3.3	6	2.5	
102	5.8	2.7	5.1	1.9	
103	7.1	3	5.9	2.1	
...	...	...	...	...	
149	6.2	3.4	5.4	2.3	
150	5.9	3	5.1	1.8	

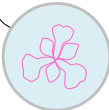


Figure 2 · A Photo Is Fundamentally a Data Matrix

- A grayscale photo is a heatmap of pixels — each pixel is one matrix entry
- Zoom in far enough and you see discrete pixel cells; 4 pixels give a 2×2 matrix

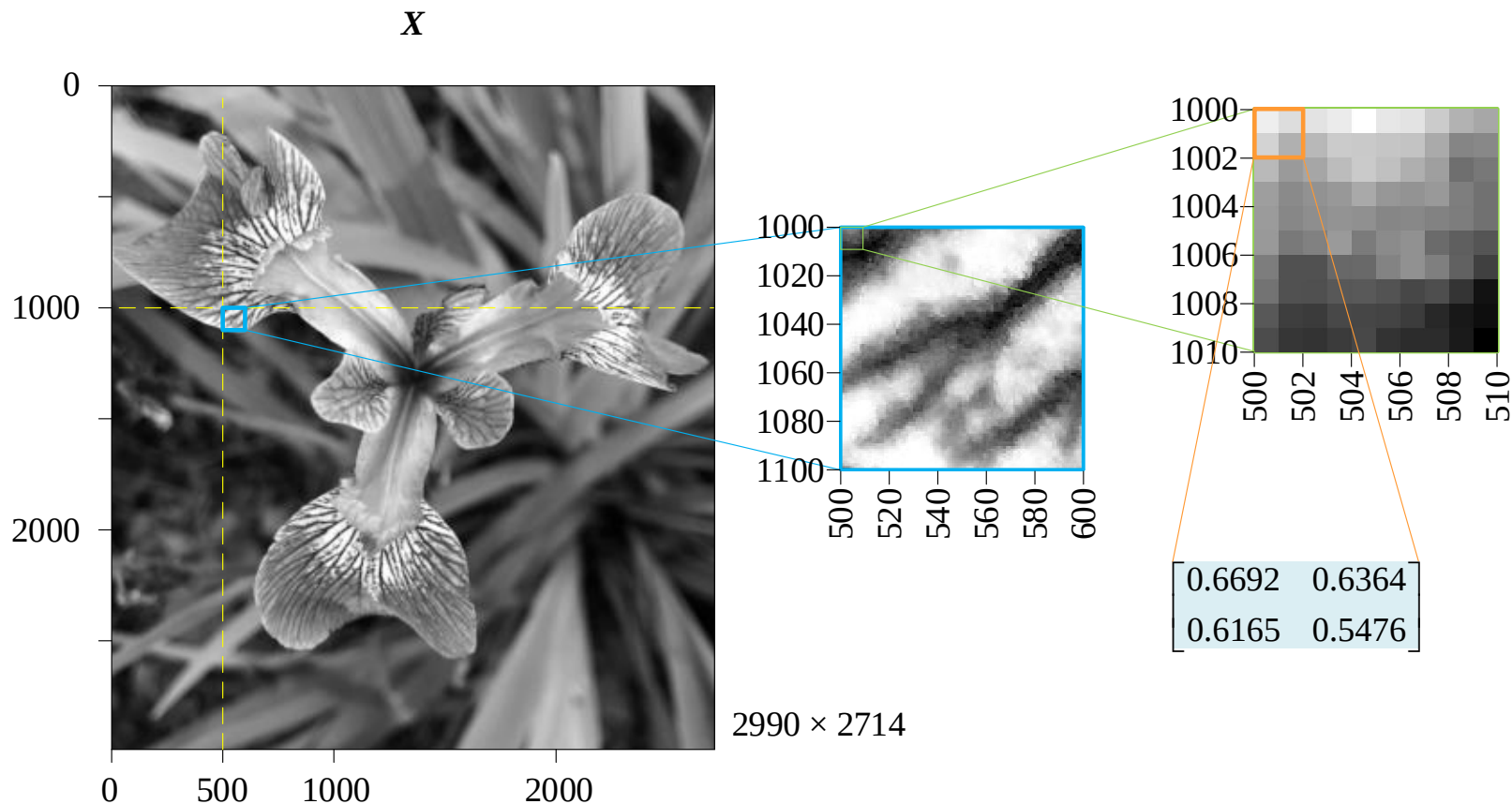
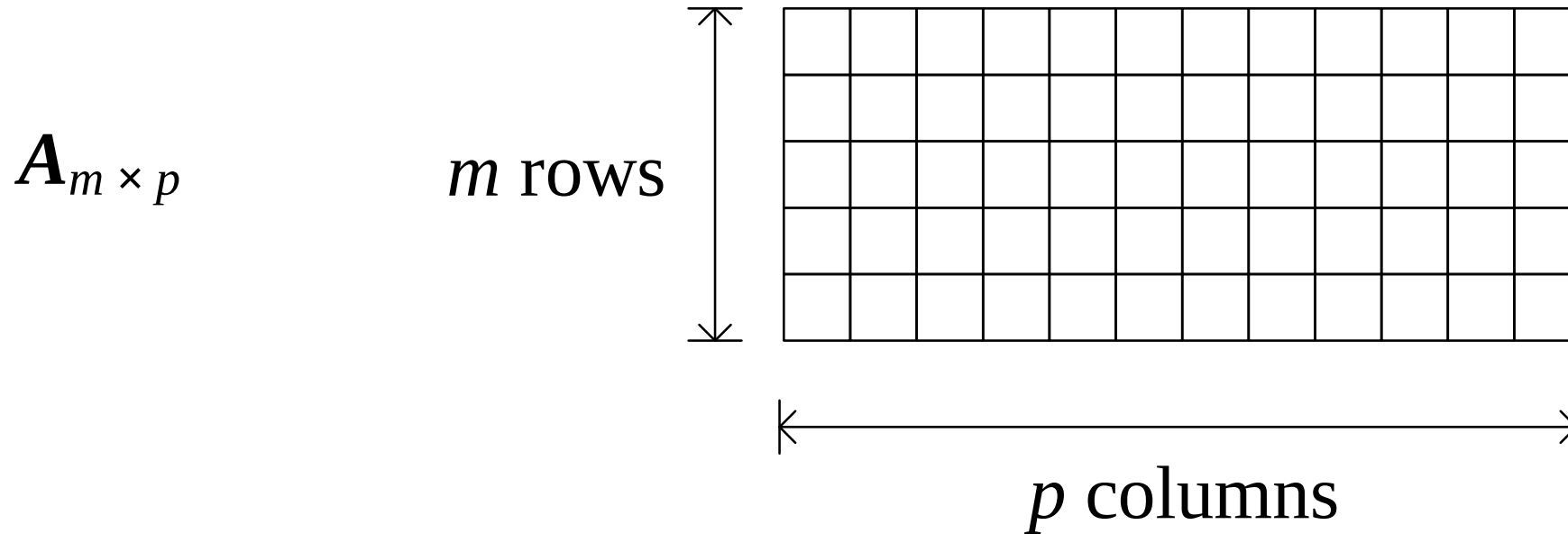


Figure 3 · The Shape of Matrix A Is Written $m \times p$

- m is the row count, p the column count; written $\mathbf{A}_{m \times p}$, with $m \times p$ entries in total
- Every entry in this book is visualized as a neat square cell



The "Growth Direction" of a Matrix: Rows Before Columns

$A_{2 \times 3}$ is like a 2-story building with 3 rooms per floor; $B_{3 \times 2}$ is 3 stories with 2 rooms per floor

- When writing a matrix's shape, the row count always comes first, then the column count — this convention hints at the matrix's "growth direction"
- A row vector is a matrix with just one row and multiple columns; a column vector has multiple rows and just one column — both are special shapes of a matrix
- A square matrix is one whose row count equals its column count

Figure 4 · The Row Index of Matrix A

- Row index i increases top to bottom, from 1 to m
- Row i of $\mathbf{A}$ is written $\mathbf{a}^{(i)}$ — e.g. row 1 is $\mathbf{a}^{(1)}$, row 2 is $\mathbf{a}^{(2)}$

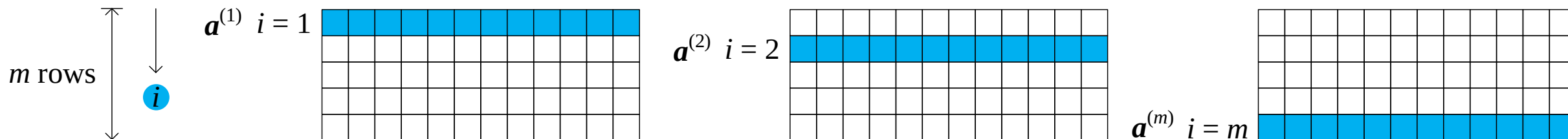
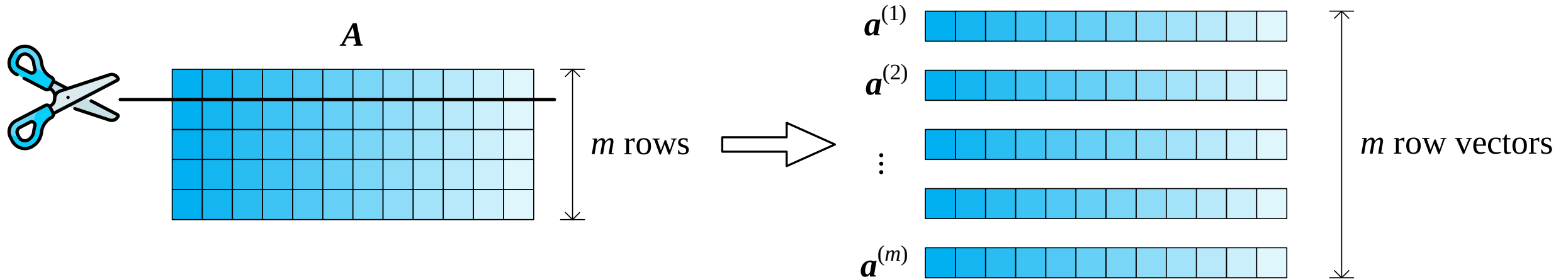


Figure 5 · Cutting Matrix A into a Set of Row Vectors

- A matrix is an ordered set of row vectors: every row is a row vector $\mathbf{a}^{(1)}, \mathbf{a}^{(2)}, \dots, \mathbf{a}^{(m)}$
- This is the block-matrix idea: splitting a large matrix into submatrices (blocks) by row and column



Example: Cutting a Matrix into Row Vectors

$$\mathbf{A} = [1,2,3; 4,5,6] = [\mathbf{a}^{(1)}; \mathbf{a}^{(2)}]$$

- The two rows of the 2×3 matrix $\mathbf{A}$ can be written as row vectors $\mathbf{a}^{(1)}=[1,2,3]$ and $\mathbf{a}^{(2)}=[4,5,6]$
- Likewise, the 3×2 matrix $\mathbf{B}=[1,4; 2,5; 3,6]$ can be written as three row vectors $\mathbf{b}^{(1)}=[1,4]$, $\mathbf{b}^{(2)}=[2,5]$, $\mathbf{b}^{(3)}=[3,6]$

Extracting Row Vectors from a Matrix with NumPy

*A[[0],:] vs. A[0,:] – same row,
different dimensionality*

- `np.random.seed(88)` fixes the random seed; `np.random.randint(0,10,(5,12))` generates a random 5×12 integer matrix
- `seaborn.heatmap()` visualizes the matrix as a heatmap; `cmap='RdYlBu_r'` sets a red-yellow-blue gradient, and `square=True` keeps each cell square
- `A[[0],:]` returns a 2D array of shape (1,12), preserving the "matrix" structure; `A[0,:]` returns a 1D array of shape (12,)
- The negative index `A[[-1],:]` extracts the last row directly, without needing to know the matrix's row count in advance

```
## 初始化
import numpy as np
import seaborn as sns

## 创建矩阵
a np.random.seed(88)
b A = np.random.randint(0, 10, (5, 12))

## 可视化
c sns.heatmap(A, cmap = 'RdYlBu_r', square = True,
               linecolor = 'w', linewidths = 0.25)

## 行索引
d a_row_1 = A[[0],:]
e A[0,:]

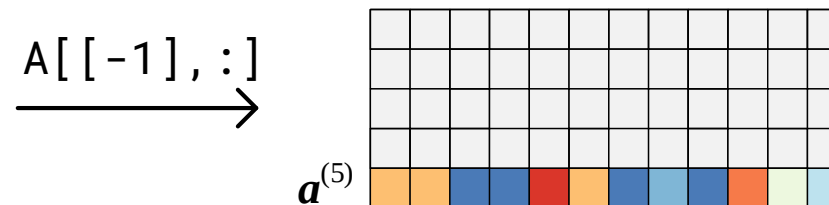
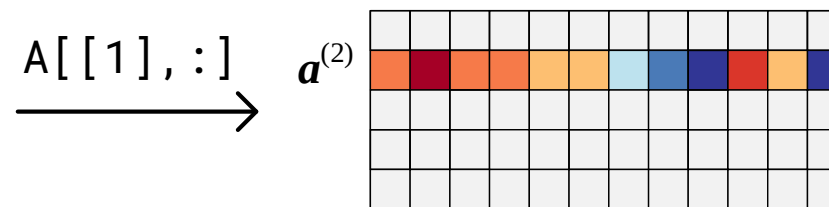
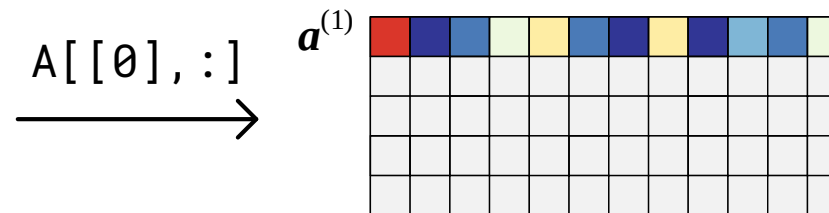
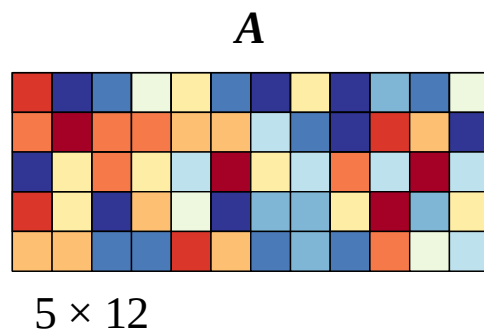
f a_row_2 = A[[1],:]

g a_row_5 = A[[-1],:]
```



Figure 6 · Extracting Row Vectors

- $A[[0],:]$ extracts row 1, $\mathbf{a}^{(1)}$; $A[[1],:]$ extracts row 2, $\mathbf{a}^{(2)}$
- $A[[-1],:]$ uses a negative index to extract the last row, $\mathbf{a}^{(5)}$



COLUMNS

Figure 7 · The Column Index of Matrix A

- Column index k increases left to right, from 1 to p
- Column k of $\mathbf{A}$ is written $\mathbf{a}_k$ — e.g. column 1 is $\mathbf{a}_1$, column 2 is $\mathbf{a}_2$

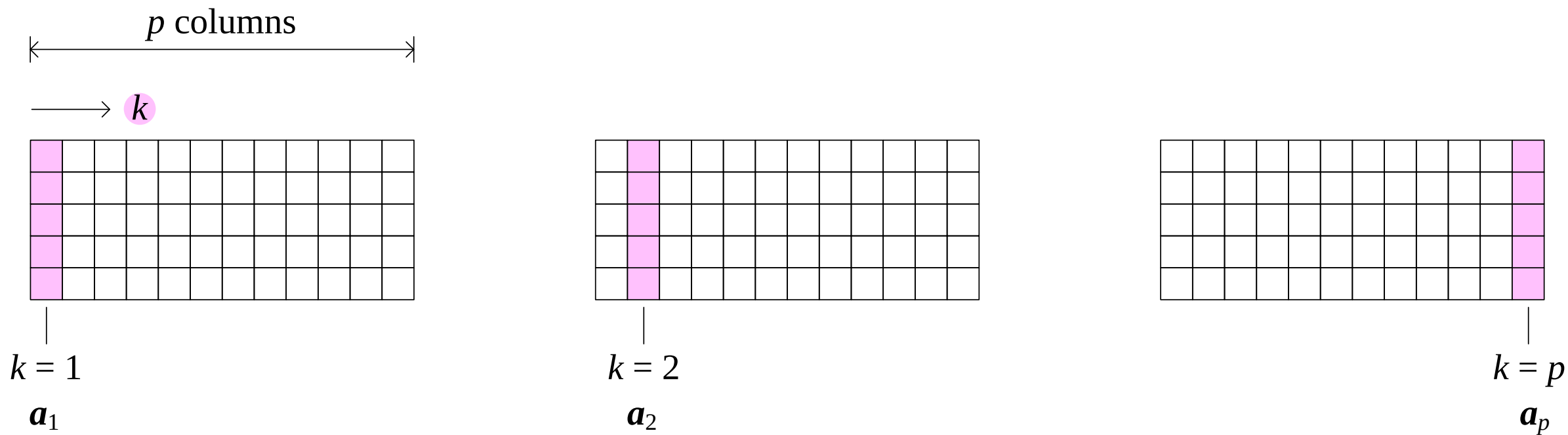
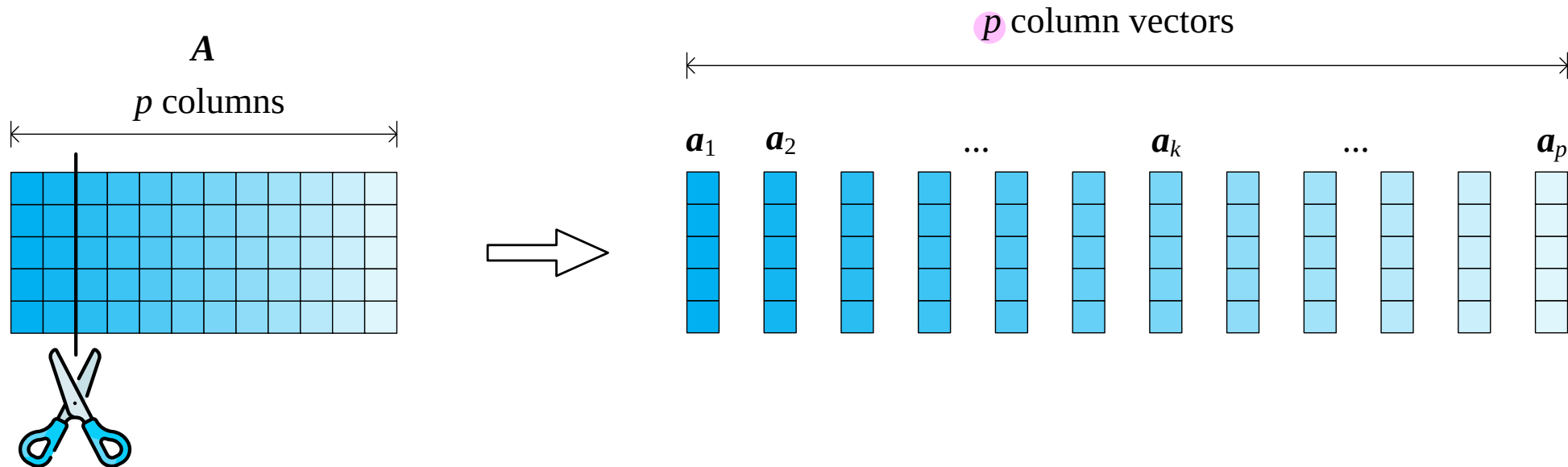


Figure 8 · Cutting Matrix A into a Set of Column Vectors

- A matrix is also an ordered set of column vectors: every column is a column vector $\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_p$
- Same block-matrix idea, only partitioned by column this time



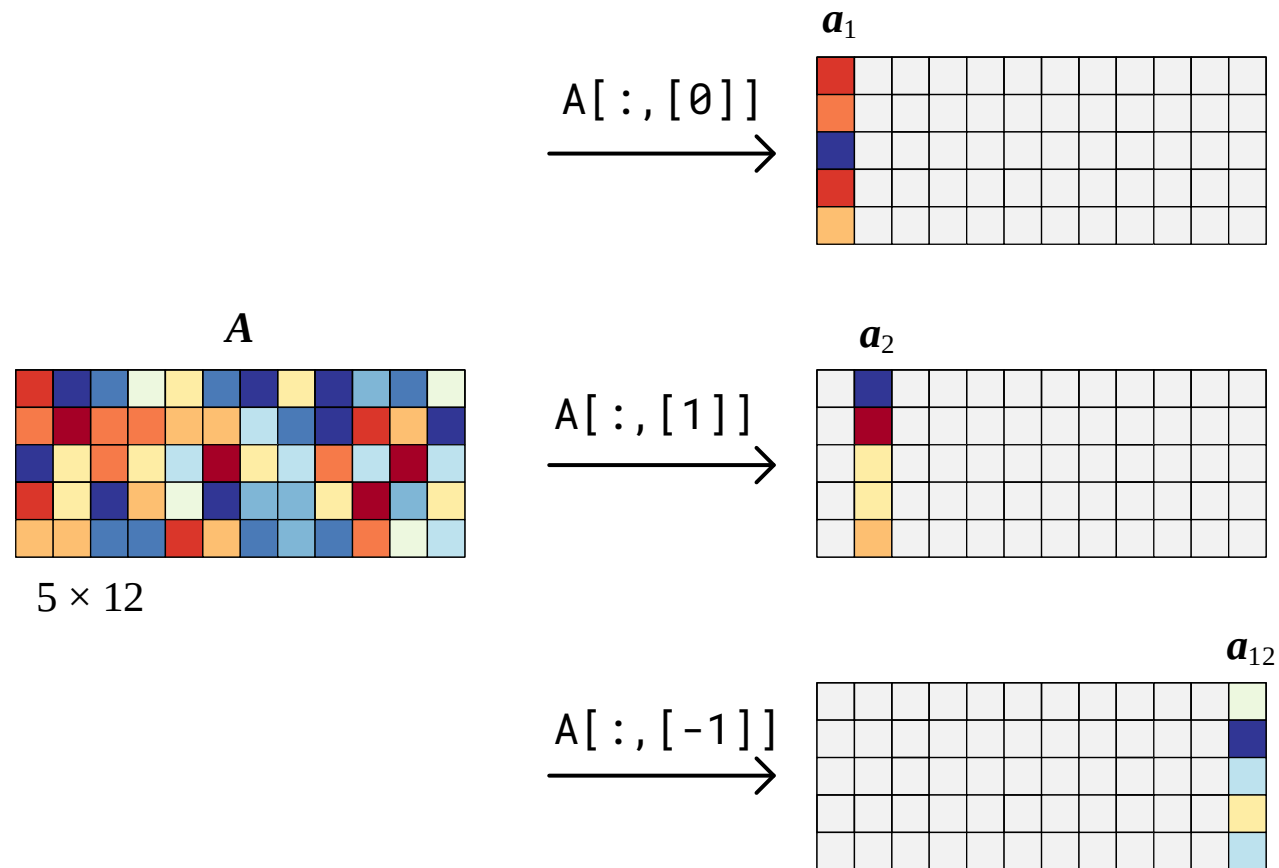
Example: Cutting a Matrix into Column Vectors

$$\mathbf{A} = [1,2,3; 4,5,6] = [\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3]$$

- The three columns of the 2×3 matrix $\mathbf{A}$ can be written as column vectors $\mathbf{a}_1=[1,4]^\top$, $\mathbf{a}_2=[2,5]^\top$, $\mathbf{a}_3=[3,6]^\top$
- Likewise, the 3×2 matrix $\mathbf{B}=[1,4; 2,5; 3,6]$ can be written as two column vectors $\mathbf{b}_1=[1,2,3]^\top$, $\mathbf{b}_2=[4,5,6]^\top$

Figure 9 · Extracting Column Vectors

- $A[:, [0]]$ extracts column 1, $\mathbf{a}_1$; $A[:, [1]]$ extracts column 2, $\mathbf{a}_2$
- $A[:, [-1]]$ uses a negative index to extract the last column, $\mathbf{a}_{12}$



Notation for a Matrix Element: $a_{i,k}$

The entry of A in row i , column k is written $a_{i,k}$

- With row and column indices in hand, we can pin down the location of any entry in a matrix
- Example: the entry of A in row 1, column 1 is $a_{1,1}$; the entry in row 2, column 3 is $a_{2,3}$

Figure 10 · Extracting Elements

- $A[0,0]$ extracts entry $a_{1,1}$; $A[2,7]$ extracts entry $a_{3,8}$
- $A[-1,-1]$ uses negative indices to extract the last entry, $a_{5,12}$

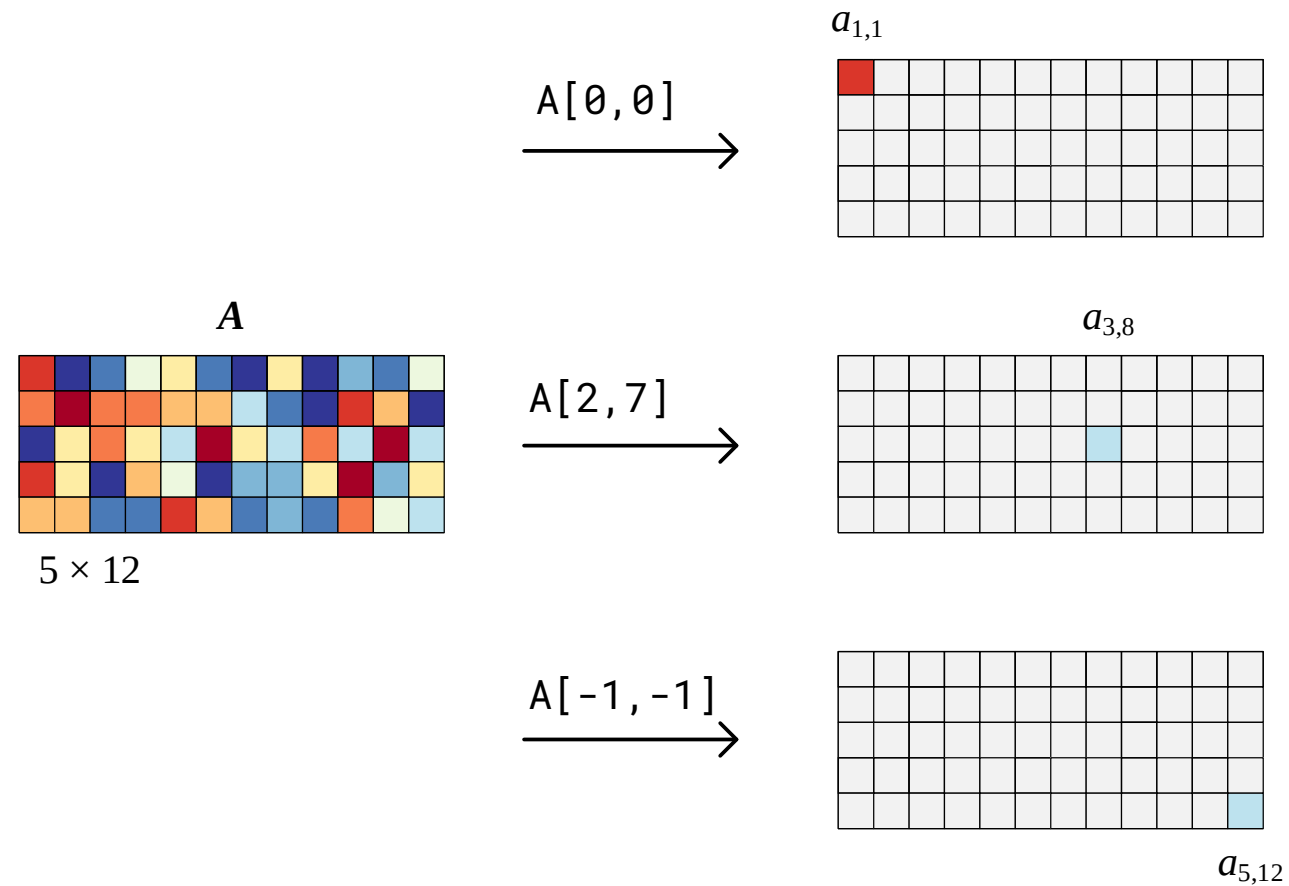


Figure 11 · The Diagonal Elements of Matrix A

- Diagonal elements share the same row and column index — the blue cells here: $a_{1,1}$, $a_{2,2}$, $a_{3,3}$
- The line through the diagonal elements is the main diagonal — the red line

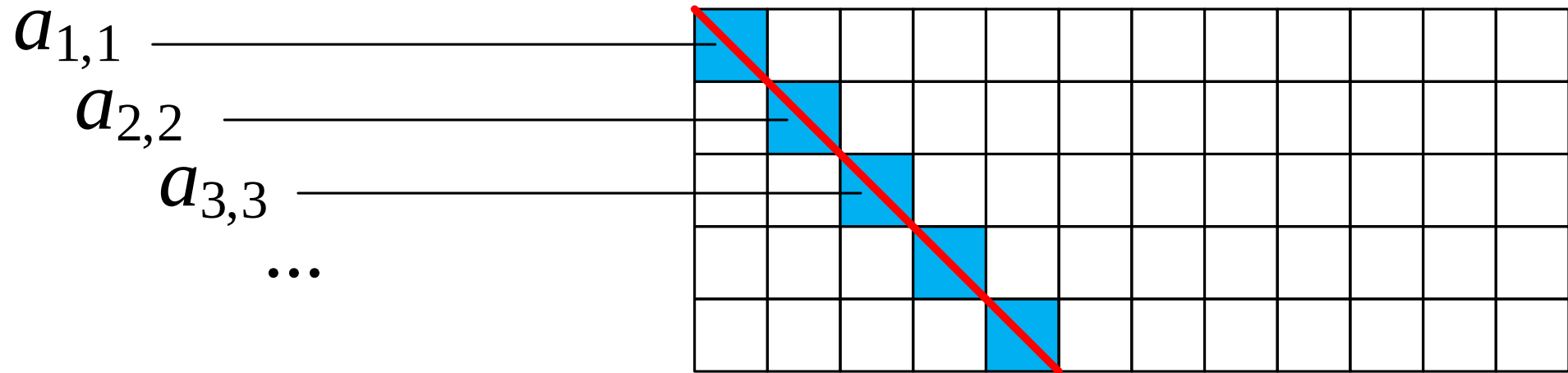


Figure 12 · Every Shape of Matrix Has Its Own Main Diagonal

- The main diagonal runs from the top-left corner at 45° toward the bottom right
- Tall, wide, square, row vector, column vector — whatever the shape, the main diagonal always exists

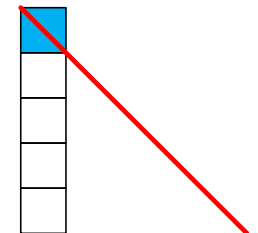
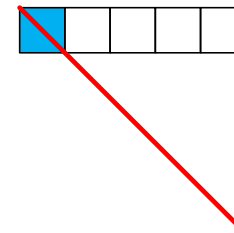
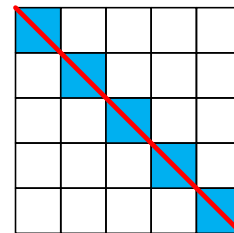
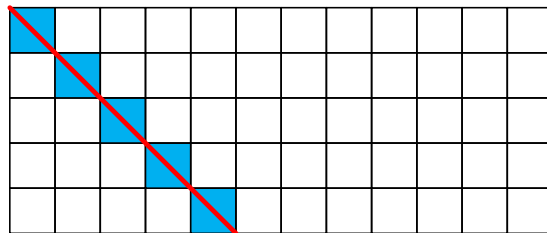
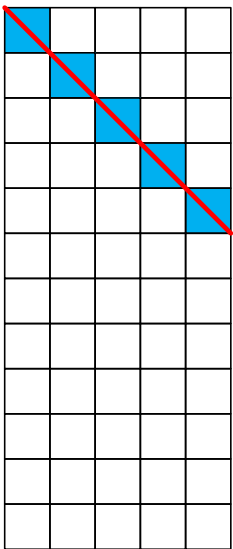
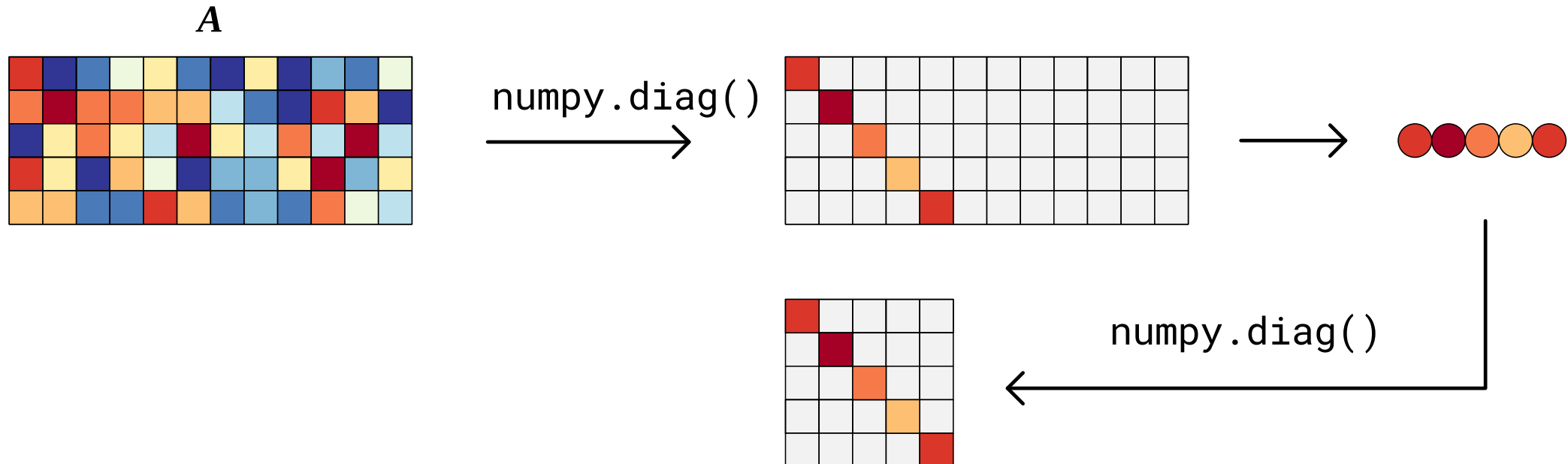


Figure 13 · Extracting or Creating a Diagonal with `numpy.diag()`

- `numpy.diag(A)` extracts the main-diagonal entries of `A` as a 1D array
- Passed a 1D array instead, it builds a diagonal matrix with zeros elsewhere



Two Matrices of the Same Shape Combine Entry by Entry

$$(A \pm B)_{i,j} = a_{i,j} \pm b_{i,j}$$

- Adding (or subtracting) two matrices **A** and **B** of the same shape means adding (or subtracting) their corresponding entries one by one
- Only matrices of exactly the same shape can be added or subtracted; the resulting matrix keeps that same shape

Matrix Addition Is Commutative and Associative

$$\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$$

- Commutative property: it doesn't matter which matrix you add first — the result is always the same
- Associative property: $\mathbf{A} + \mathbf{B} + \mathbf{C} = \mathbf{A} + (\mathbf{B} + \mathbf{C}) = (\mathbf{A} + \mathbf{B}) + \mathbf{C}$ — summing any two matrices first, then adding the third, always gives the same result
- The same rules apply to matrix subtraction

Zero Matrix $\mathbf{O}$: All Entries Equal to 0

$$\mathbf{A} + \mathbf{O} = \mathbf{O} + \mathbf{A} = \mathbf{A}, \quad \mathbf{A} - \mathbf{A} = \mathbf{O}$$

- This book uses $\mathbf{O}$ to denote the matrix whose entries are all 0 — the zero matrix
- Adding a zero matrix of matching shape to any matrix leaves it unchanged; subtracting a matrix from itself gives the zero matrix

Scalar Multiplication: Scaling Every Entry by a Number

$$k\mathbf{A} = [k \cdot a_{i,j}]$$

- Scalar multiplication means multiplying every entry of a matrix by a single scalar k
- Matrix $\mathbf{A}$ has shape $m \times p$; after scalar multiplication the shape is unchanged, still $m \times p$ — every entry is simply scaled up or down
- In particular, when $k=0$, the result is the zero matrix $\mathbf{O}$, still of shape $m \times p$

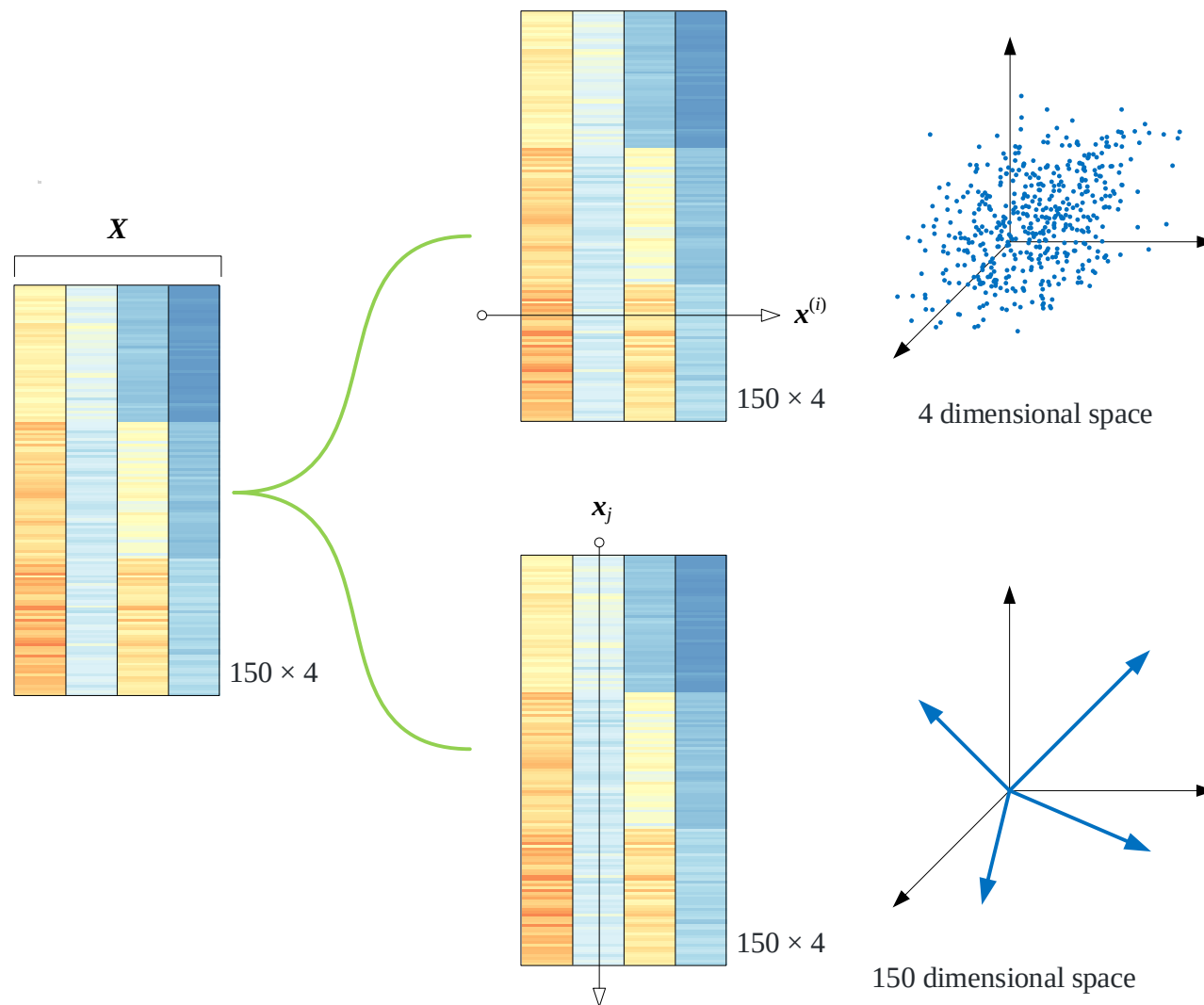
From Scalar to Matrix: the Order of a Tensor

0th-order tensor = scalar, 1st-order tensor = vector, 2nd-order tensor = matrix

- A tensor is a multidimensional array in mathematics, describing numerical data across several dimensions; its order determines how much complexity it can express
- Tensors of order 3 and above are generally called multidimensional arrays — a color image (height × width × channel) is a 3rd-order tensor
- A D -dimensional space is written $\mathbb{R}^D$: a 2D plane is $\mathbb{R}^2$, Iris's 4 features live in $\mathbb{R}^4$, its 150 samples in $\mathbb{R}^{150}$

Figure 14 · Viewing the Iris Data by Row and by Column

- By row: each row $\mathbf{x}^{(i)}$ is a point of 4 features, living in a 4D space
- By column: each column $\mathbf{x}_j$ holds all 150 samples' values on one feature, a vector in 150D space



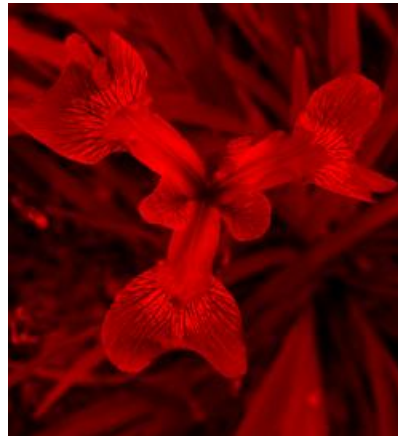
From Pixel to Photo: Tensors of Different Orders

An RGB color $c = [R, G, B]^T$ is a 1st-order tensor

- 0th-order tensor example: the grayscale value of a single pixel in a black-and-white photo, say 0.5
- 1st-order tensor example: an RGB color, written as a column vector $[R, G, B]^T$
- 2nd-order tensor example: a black-and-white photo, with two dimensions — height H and width W — written $\mathbb{R}^{H \times W}$; a 2-row, 3-column matrix is likewise 2nd-order

Figure 15 · A Color Photo Decomposed into R, G, B Channels

- A color photo has R, G, B channels — a 3rd-order tensor, written $\mathbb{R}^{H \times W \times C}$
- Each channel is its own 2nd-order tensor; add a time dimension for video, and it's 4th-order



Tensors: A Core Tool of Modern Machine Learning

Scalar $\rightarrow$ Vector $\rightarrow$ Matrix $\rightarrow$ Tensor: data representation grows step by step

- A neural network's weight matrices, an image's pixel channels, and word-embedding vectors in NLP all exist in the form of tensors
- Tensors let operations like matrix multiplication and convolution run efficiently and in parallel on hardware such as GPUs — the key to deep learning at scale
- Across computer vision, natural language processing, and time-series analysis, tensors remain an indispensable foundation of modern AI systems

Key Terms in This Section

Matrix *A two-dimensional array arranged in rows and columns*

Row / Column Index *Row index i runs top to bottom, column index k left to right, both starting at 1*

Block Matrix *A large matrix partitioned by rows/columns into smaller submatrices (blocks)*

Diagonal Element / Main Diagonal *Entries whose row and column index match, and the line through them*

Scalar Multiplication *The product kA of a scalar k and a matrix A*

Shape ($m \times p$) *m is the row count, p the column count, written $A_{m \times p}$*

Row / Column Vector *Special matrix shapes: one row with many columns, or many rows with one column*

Matrix Entry *Written $a_{i,k}$, where i is the row index and k the column index*

Zero Matrix *A matrix whose entries are all 0, written O*

Tensor *A multidimensional array; scalar = 0th-order, vector = 1st, matrix = 2nd*

GO FURTHER

Companion Code and Further Reading

Code 1 · LA_02_01_01.ipynb

Creating a matrix with NumPy; extracting rows, columns, and elements

Further Reading · LA_02_01_02.ipynb

NumPy implementations of matrix addition, subtraction, and scalar multiplication — self-study

Recommended Reading · The Power of Matrices, Coding Made Easy

Source of this section's figures — a deeper look at matrices, photos, and data

Open-source Repository

Code and image assets for the whole book: <https://github.com/Visualize-ML>

Questions Q1 – Q10

Q1 Learn to draw a heatmap with `seaborn.heatmap()`.

Q3 How do you build a Pandas DataFrame from a NumPy 2D array?

Q5 From the array in Q4, extract row 1, row 5, and the last row (using two indexing styles, and compare the resulting dimensions); column 1, column 5, and the last column; the elements at rows 1, 2, 3, 8 in the matching column; and all the main-diagonal elements.

Q7 Study NumPy's broadcasting mechanism on your own.

Q9 Learn the following NumPy functions and explain what each does: `flatten`, `column_stack`, `row_stack`, `flip`, `flipr`, `flipud`, `reshape`.

Q2 What is a Pandas DataFrame? How does it relate to, and differ from, a NumPy array?

Q4 Generate an 8-row, 18-column random array and visualize it with `seaborn.heatmap()`.

Q6 Generate two random 3-row, 8-column arrays, and compute their sum and difference.

Q8 Generate an 8×8 random array and compute the max, min, and mean of each column.

Q10 Study the Hadamard product (element-wise product) of matrices on your own.

Questions Q11 – Q20

Q11 Create a 4×4 matrix A with NumPy, and verify that its main-diagonal entries match the output of `numpy.diag(A)`.

Q13 Write a function `make_block_cols(A)` that splits any matrix A into its column vectors and returns them as a list.

Q15 Write a function `is_square(A)` that checks whether a matrix is square, and test it on randomly generated matrices.

Q17 Generate three random matrices A, B, C of the same shape, and write code to verify the associative property $(A+B)+C = A+(B+C)$.

Q19 Use `seaborn.heatmap()` to visualize matrix A alongside the scalar-multiplied results $2A$ and $-1A$, and observe how the coloring changes.

Q12 Write a function `make_block_rows(A)` that splits any matrix A into its row vectors and returns them as a list.

Q14 Generate a random 6×6 integer matrix and write code to extract every entry off the main diagonal (hint: `numpy.eye()` can help build a mask).

Q16 Generate two random 5×5 matrices A and B of the same shape, and write code to verify the commutative property $A+B=B+A$.

Q18 Write a function `scalar_multiply(A, k)` that implements matrix scalar multiplication by hand (without using the `*` operator), and compare it against $A*k$.

Q20 Write code that uses `numpy.diag()` to build a 5×5 diagonal matrix from the 1D array `[1,2,3,4,5]`, and visualize it as a heatmap.

Questions Q21 – Q30

Q21 Generate a random 10×3 matrix simulating 10 samples with 3 features, and extract the first sample and the first feature by row and by column.

Q23 Load a color photo with NumPy (or simulate an $H \times W \times 3$ array with random numbers), and extract and visualize its R, G, B channels separately (as in Figure 15).

Q25 Load the Iris dataset with Pandas, convert it to a NumPy 2D array, and write code to extract all the row vectors belonging to the Setosa species.

Q27 Generate a random matrix, count how many entries exceed the matrix's average value, and highlight those positions in a heatmap.

Q29 Write a function `block_col_matrix(vectors)` that takes a set of same-shape column vectors and reassembles them into a matrix (hint: `numpy.hstack()`).

Q22 Write a function `tensor_order(x)` that takes a NumPy array and returns its order (i.e. `x.ndim`); test it on a scalar, a vector, a matrix, and a 3D array.

Q24 Write code verifying that reshaping an $H \times W \times 3$ color image array to 1D and back to its original shape leaves the image content unchanged.

Q26 Write a function `matrix_summary(A)` that reports the shape of `A`, the sum of its main-diagonal entries (its trace), and the sum of all its entries.

Q28 Write a function `block_row_matrix(vectors)` that takes a set of same-shape row vectors and reassembles them into a matrix (hint: `numpy.vstack()`).

Q30 Bringing it together, write a small program that, given any matrix, prints its shape, its list of row vectors, its list of column vectors, its main-diagonal entries, and the result of multiplying it by the scalar 2.

10 Prompts to Explore Further (1/2)

- 1 Explain why a matrix's shape is written "rows first, then columns," and how that convention relates to the matrix's "growth direction."
- 2 Use an everyday example to explain the block-matrix idea — like splitting a spreadsheet into smaller tables by row or by column.
- 3 Prove that adding a zero matrix of matching shape to any matrix leaves it unchanged; what practical use does this property have in programming?
- 4 Why does NumPy index from 0 while mathematical convention indexes matrix rows and columns from 1? What common bugs does this mismatch tend to cause?
- 5 Explain why the main diagonal matters in matrix operations, such as computing the trace or checking whether a matrix is symmetric.

10 Prompts to Explore Further (2/2)

6

Design a small Python experiment that verifies matrix addition's commutative and associative properties hold for randomly generated matrices.

7

Explain the geometric meaning of scalar multiplication by $k=0$ producing a zero matrix — what actually happens when a vector or matrix is multiplied by 0?

8

Explain how a tensor's "order" grows with data complexity, from scalar to vector to matrix, and on to color images and video.

9

Weight matrices in neural networks, image pixel channels, and word-embedding vectors are all tensors — give a concrete machine learning model example showing what role tensors play.

10

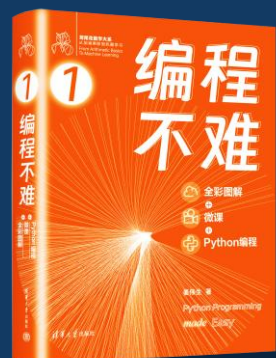
If you wanted to visualize a 4th-order tensor, such as video data, what approach would you take? Give at least two possible strategies.

数学

数学基础

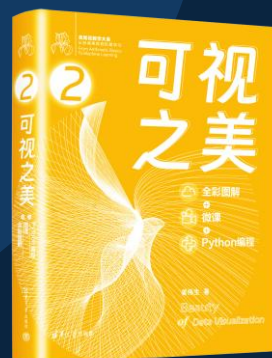
线性代数

概率统计

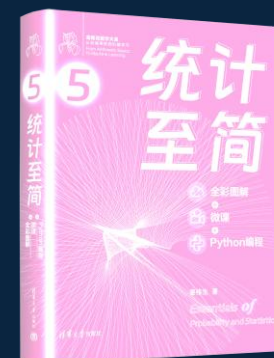


Python编程

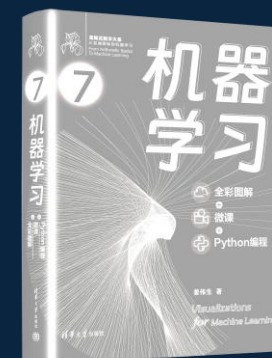
工具



数据可视化



数据分析
图和网络



回归、降维
分类、聚类

实践

